

NEUKRAG: NEUROMORPHIC KG–RAG WITH SMALL LLMs for Hardware–Algorithm Co-Design

Ramakrishnan Kannan*, Ashish Gautam*, Robert Patton*,

Nicholas D Haas*, Todd Thomas*, James B. Aimone[&], Thomas Potok*

*Oak Ridge National Laboratory, Oak Ridge, TN, USA

[&]Sandia National Laboratories, Albuquerque, NM, USA.

{kannanr, gautama, pattonrm, haasnq, thomastm}@ornl.gov, jbaimon@sandia.gov, potokte@ornl.gov

Abstract—We introduce a scalable system for constructing scientific Knowledge Graphs (KGs) from neuromorphic computing literature using open-source, lightweight Large Language Models (LLMs), and for leveraging these graphs as structured retrieval substrates in a GraphRAG framework. Our pipeline integrates extraction, merging, canonicalization, and clustering to transform unstructured text into a consistent, high-quality KG. We deploy models such as `gpt-oss-120b`, `llama3.3-70b`, `granite4-small-h` and `gpt-oss-20b` as LLM backends in a cost-effective distributed processing architecture. Applied to 2,396 documents, the system extracted over 921,545 triples with a 100% query completion rate. We show that the resulting KG supports multi-hop reasoning, explicit provenance, and robust retrieval that overcomes the limitations of standard vector-based RAG. Within a neuromorphic co-design context, the GraphRAG layer provides substantially deeper and more hardware-aware responses than naive LLMs, capturing relationships across algorithms, circuits, devices, and architectures. These results demonstrate that combining open LLMs with structured scientific knowledge yields a practical and explainable foundation for accelerating research in neuromorphic computing and other technically complex domains.

I. INTRODUCTION

Neuromorphic computing is an inherently multidisciplinary field that aims to develop information-processing systems inspired by biological brains, leveraging sparse spiking communication, co-located memory and computation, asynchronous dynamics, and adaptive behavior [1], [2], [3], [4], [5]. These characteristics allow neuromorphic platforms to operate with exceptionally low power consumption, low latency, and real-time responsiveness for tasks such as sensory processing, robotics, and on-device learning. Beyond enabling brain-like artificial intelligence, neuromorphic principles also provide a foundation for next-generation in-memory computing architectures that can substantially reduce the energy footprint of modern AI and scientific computing workloads [6], [7], [8]. Contemporary neuromorphic systems integrate spiking neural networks (SNNs), mixed-signal neuron and synapse circuits, custom interconnect fabrics, and a variety of emerging memory

technologies into a tightly coupled hardware–software stack [5], [9], [10], [11], [12], [13]. Within this stack, algorithmic choices, circuit topologies, device characteristics, and system-level design decisions interact in complex and often non-linear ways, such that constraints or innovations at one layer propagate through and shape opportunities across the others.

Recent works have emphasized that neuromorphic progress depends on cross-domain synthesis. Realizing next-generation neuromorphic systems requires iterative co-design across neuroscience, circuit design, materials science, computer architecture, and machine learning [1]. A typical workflow spans high-level SNN modeling, hardware-aware simulation with device nonidealities, FPGA prototyping, mixed-signal circuit validation, and system-level software integration [14]. At each stage, researchers must synthesize heterogeneous knowledge—for example, relating a synaptic learning rule to device variability, or connecting circuit-level noise characteristics to algorithmic robustness. However, this information is distributed across thousands of publications, simulation repositories, and device characterizations, making it difficult to track assumptions, compare methodologies, or identify cross-domain dependencies. As the neuromorphic design space grows, these challenges increasingly limit systematic exploration and principled innovation.

Large Language Models (LLMs) offer new opportunities to accelerate cross-domain synthesis by analyzing large corpora of scientific literature and surfacing relevant connections. Nevertheless, free-form LLM outputs remain difficult to trust in scientific workflows because they lack structured grounding, obscure provenance, and may produce hallucinated or unverifiable claims. Retrieval-Augmented Generation (RAG) partially mitigates these issues by conditioning LLM responses on retrieved documents, but standard vector-based retrieval still struggles with the multi-hop, relational reasoning required in neuromorphic co-design. Vector similarity captures local semantic proximity but cannot represent explicit relationships—such as device $\rightarrow$ circuit $\rightarrow$ learning rule $\rightarrow$ performance—as traversable structures. Consequently, queries involving compositional reasoning or evidence scattered across multiple papers remain challenging.

Knowledge Graphs (KGs) provide a principled mechanism for representing scientific knowledge in a structured, explainable form. By encoding entities and relationships as triples, a KG exposes the logical and scientific connections that underpin neuromorphic research. This structure supports multi-hop reasoning, canonicalization and deduplication across sources, and transparent provenance tracking. When

An award of computer time was provided by the ASCR Leadership Computing Challenge (ALCC) program. This research used resources of the Oak Ridge Leadership Computing Facility, a DOE Office of Science User Facility. This manuscript has been authored by UT-Battelle, LLC under Contract No. DE-AC05-00OR22725 with the US Department of Energy. The United States Government retains and the publisher, by accepting the article for publication, acknowledges that the United States Government retains a non-exclusive, paid-up, irrevocable, world-wide license to publish or reproduce the published form of this manuscript or allow others to do so, for United States Government purposes. The Department of Energy will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan (<http://energy.gov/downloads/doe-public-access-plan>).

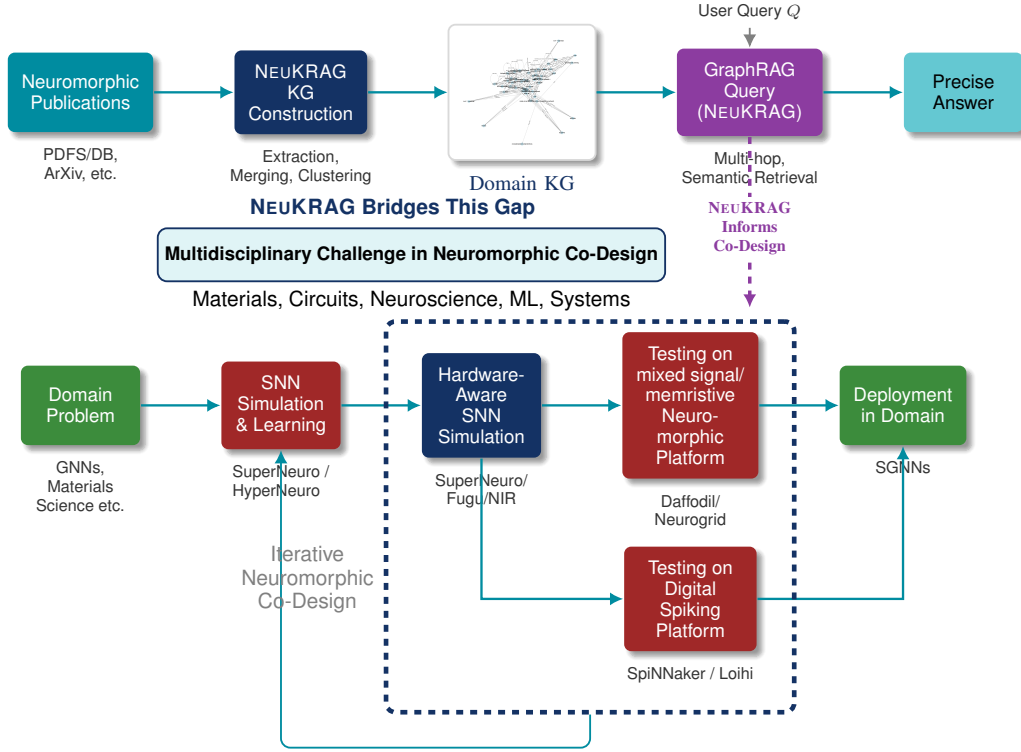


Fig. 1. Bird's eye view of the NEUROMORPHIC KG-RAG WITH SMALL LLMs (NEUKRAG)

combined with LLMs, KGs enable a form of structured RAG in which retrieval leverages both semantic similarity (via vectors) and graph topology (via k-hop expansion), improving interpretability and faithfulness in generated responses.

To operationalize these capabilities, we introduce NEUROMORPHIC KG-RAG WITH SMALL LLMs (NEUKRAG), a framework designed to transform neuromorphic literature into a structured, queryable knowledge substrate. In this paper, we focus specifically on the automated construction of a domain-specific KG that captures entities, mechanisms, devices, and design relationships central to neuromorphic computing. Unlike unstructured vector retrieval, the NEUKRAG explicitly models interdependencies, enabling LLMs to traverse coherent chains of scientific reasoning rather than relying solely on keyword similarity. This structured supervision yields more accurate, explainable, and context-aware responses to complex engineering queries, effectively bridging cross-domain gaps in neuromorphic co-design. Our system uses lightweight LLMs—including gpt-oss-120b, llama3.3-70b, gpt-oss-20b, and ibm/granite4-small-h—served via vLLM and Ollama. Through prompt specialization and model routing, the pipeline efficiently extracts structured triples from unstructured text at low cost. The resulting KG serves as the structured retrieval substrate for a GraphRAG query layer that integrates vector search, graph traversal, and LLM-based reasoning to support complex multi-hop scientific queries.

This work makes the following contributions:

- We develop an end-to-end KG construction pipeline comprising extraction, merging, post-processing, deduplication, and clustering stages.

- We propose a GraphRAG query procedure that integrates vector similarity search with graph traversal (k-hop expansion) and LLM-based reasoning.
- We report detailed runtime and scaling analyses from processing 2,396 neuromorphic publications, yielding over 921,545 structured triples.
- We demonstrate the system's utility through visualizations and evaluations of GraphRAG outputs on complex neuromorphic design queries.

II. BACKGROUND AND RELATED WORK

A. Information Extraction with LLMs

Information Extraction (IE) has traditionally relied on supervised models, but recent work has demonstrated the few-shot and zero-shot capabilities of LLMs for triple extraction [15]. OpenIE approaches [16] extract surface-level relations, while ontology-based methods map text to predefined schemas. LLM-driven extraction frameworks such as [17] show that prompt-based generative models can flexibly handle varied sentence structures. However, neuromorphic text poses unique challenges—dense terminology, nested clauses, and implicit causal relationships—requiring multi-stage refinement to reduce noise and hallucinations. Our system adopts prompt-based extraction with lightweight open-source LLMs and introduces a multi-pass pipeline designed specifically for neuromorphic publications. Constructing KGs from scientific literature presents unique challenges due to domain-specific terminology and complex sentence structures. Previous efforts like the Open Research Knowledge Graph (ORKG) [18] rely on crowdsourcing or semi-automated methods. Automated

approaches often use BERT-based models [19]. We advance this by employing generative LLMs for flexible schema-free extraction followed by clustering-based canonicalization.

B. Graph-Augmented RAG

Graph-augmented RAG enhances retrieval by using KGs to provide structural context. Methods like G-Retriever [20] and GraphRAG [21] integrate graph traversal with vector search. Our approach aligns with these by using a k-hop subgraph retrieval mechanism but emphasizes the use of open-source models for both construction and reasoning, ensuring reproducibility and data privacy.

C. Canonicalization and Deduplication

Entity resolution and canonicalization are critical for KG quality. Techniques range from string similarity metrics to embedding-based clustering [22]. We implement a hybrid approach using fuzzy string matching (dedupe) and community detection (Louvain clustering) on the graph structure to merge synonymous entities and reduce noise.

D. Neuromorphic Co-design

Neuromorphic computing relies on a cyclical algorithm–hardware co-design process in which models are iteratively shaped by domain requirements and hardware constraints. Initial exploration typically occurs in portable simulators such as SuperNeuro [23], Fugu [24], or NIR [25], enabling the development of SNN architectures, learning rules, and coding strategies without hardware limitations.

These algorithmic designs are then specialized for specific platforms. Hardware-aware configurations of SuperNeuro [26], [27] emulate quantization, timing precision, memory limits, and routing behavior. Promising solutions are next deployed on the NeuroCoreX FPGA emulator for cycle-accurate latency and energy assessment [28], and finally validated on the Daffodil memristive platform to characterize device stochasticity, endurance, and drift [29]. Similar flows apply when porting Fugu or NIR programs to large-scale mixed-signal systems such as Neurogrid [30] or digital platforms like SpiNNaker or Loihi [5].

Because each stage introduces new assumptions and constraints, a unifying representation is needed. A Knowledge Graph (KG) provides this by connecting algorithms, devices, architectures, and performance metrics, enabling traceability, informed design decisions, and systematic exploration of the neuromorphic design space.

III. NOTATIONS

Table I summarizes the mathematical notations used throughout this paper to describe the Knowledge Graph construction and retrieval processes.

IV. NEUROMORPHIC

KG–RAG WITH SMALL LLMs (NEUKRAG)

The NEUKRAG system implements a modular pipeline for constructing Knowledge Graphs from neuromorphic publications as summarized in Algorithm 1. The pipeline is orchestrated via a central command-line interface and consists

TABLE I
SUMMARY OF NOTATIONS

Symbol	Description	Dimension/Type
$G = (E, R)$	Knowledge Graph with entities and relations	Graph
E	Set of unique entities	Set
R	Set of relation types	Set
h, t	Head and tail entities ($h, t \in E$)	Entity
r	Relation connecting head and tail ($r \in R$)	Relation
$\mathcal{T}$	Set of triples $\{(h, r, t) \mid h, t \in E, r \in R\}$	Set
$\phi(\cdot)$	Embedding function to vector space	Function
d	Embedding space dimension	Integer
$\mathbf{e}$	Entity vector ($\mathbf{e} \in \mathbb{R}^d$)	Vector
Q	Natural language query	String
$\mathbf{q}$	Query vector ($\mathbf{q} \in \mathbb{R}^d$)	Vector
$\text{sim}(\cdot, \cdot)$	Similarity function (cosine)	$\mathbb{R}^d \times \mathbb{R}^d \rightarrow \mathbb{R}$
N_{entry}	Entry nodes for traversal	Set $\subset E$
θ	Similarity threshold	Scalar
k	Graph traversal hops	Integer
$\mathcal{T}_{\text{sub}}$	Retrieved triples subgraph	Set $\subset \mathcal{T}$
d_τ	Document ID for triple τ	String
$\text{Score}(\cdot)$	Document relevance function	Function
M	Number of top documents	Integer
K	Number of top triples	Integer

of sequential stages: extraction, merging, post-processing, deduplication, clustering, embedding, and ontology generation.

A. Pipeline Overview

The high-level workflow processes documents from a MongoDB collection, orchestrating the flow of data through local file storage and processing stages.

B. Extraction

The pipeline begins with the extraction phase, where raw text documents are parsed to identify semantic triples consisting of a head entity, a relation, and a tail entity. This process iterates over the document corpus, applying a set of predefined prompts designed to elicit specific types of knowledge. To ensure robustness and scalability, the system employs a multi-model client that queries various Large Language Models (LLMs) in parallel. This architecture supports distribution across multiple inference servers (e.g., vLLM, Ollama) and includes fault-tolerance mechanisms such as automatic regeneration with temperature adjustment for malformed responses. To support long-running jobs and incremental updates, the system implements a checkpointing mechanism. For each document d , the set of remaining models M_{rem} is calculated by excluding those that have already successfully processed the document ($\text{GetProcessed}(d)$), ensuring that only missing extractions are performed upon resumption. The output of this stage is a collection of JSONL files containing the extracted triples along with their provenance metadata.

C. Merging

Following extraction, the system consolidates the disparate sets of triples generated from different prompts and models into a unified dataset. This merging process aggregates entities and predicates, assigning unique identifiers to each distinct element. The result is a comprehensive set of global tables

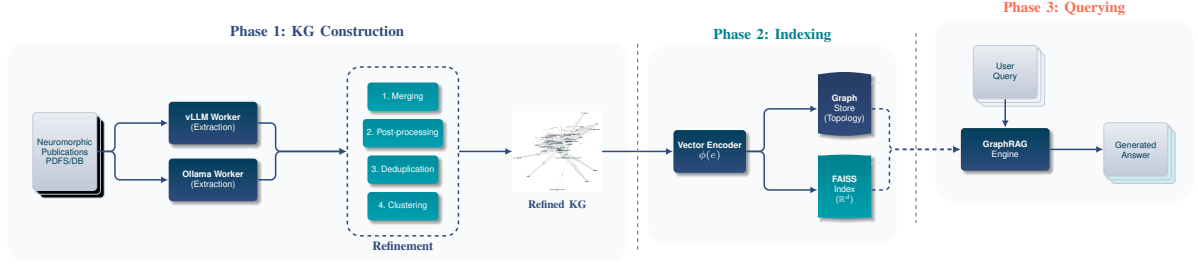


Fig. 2. Overview of the NEUROMORPHIC KG-RAG WITH SMALL LLMs (NEUKRAG)

Algorithm 1: Knowledge Graph Construction Pipeline

Input : Publications D , Config C
Output : Graph G , Ontology O , Embeddings E
 // Step 1: Save Inputs
 1 Fetch D from MongoDB and save locally;
 // Step 2: Extraction
 2 Init models M and prompts P from C ;
 3 **foreach** prompt $p \in P$ **do**
 4 **foreach** doc $d \in D$ **do**
 5 $text \leftarrow \text{ExtractText}(d)$;
 6 $M_{rem} \leftarrow M \setminus \text{GetProcessed}(d)$;
 7 **if** $M_{rem} \neq \emptyset$ **then**
 8 $Res \leftarrow \text{ParallelExtract}(text, p, M_{rem})$;
 9 **foreach** model m , triples $T_m \in Res$ **do**
 10 Normalize and save T_m to JSONL;
 // Step 3: Merge & Post-process
 11 $T_{all} \leftarrow \text{MergeTriples}(\text{JSONL files})$;
 12 $T_{clean} \leftarrow \text{CleanEntities}(T_{all})$;
 // Step 4: Deduplication
 13 $E_{dup} \leftarrow \text{FindDuplicates}(T_{clean}, C.dedupe)$;
 14 $G \leftarrow \text{MergeNodes}(T_{clean}, E_{dup})$;
 // Step 5: Clustering & Embedding
 15 $G \leftarrow \text{FilterClusters}(G, C.clustering)$;
 16 $E \leftarrow \text{GenerateEmbeddings}(G.nodes)$;
 17 Build FAISS Index from E ;
 // Step 6: Ontology
 18 $O \leftarrow \text{GenerateOntology}(G)$;
 19 Validate O ;
 20 **return** G, O, E

representing the merged entities, documents, and predicates, which serves as the foundation for subsequent refinement steps.

D. Post-Processing

To ensure data quality, a normalization layer applies a series of cleaning rules to the merged entity names and relations. These rules, defined in the system configuration, target common artifacts such as extraneous quotes, special characters, and system-specific prefixes. The goal of this stage is to standardize the textual representation of graph elements, thereby improving the accuracy of downstream deduplication and clustering tasks.

E. Deduplication

Entity resolution is handled through a deduplication module that identifies and resolves duplicate entities and triples. This process utilizes fuzzy string matching with a configurable similarity threshold to detect near-duplicate strings that likely refer to the same concept. When duplicates are identified, they are merged, and all references in the associated triples are updated to point to the canonical entity.

F. Clustering

Semantic consolidation is achieved via a clustering stage that analyzes the graph structure to detect communities. By constructing a network representation of the triples and applying the Louvain community detection algorithm, the system identifies groups of densely connected nodes. The graph is then refined by filtering out small, isolated clusters that fall below a specified size threshold, effectively reducing noise and focusing the graph on the most significant semantic structures.

G. Embedding and Indexing

To support Graph Retrieval-Augmented Generation (GraphRAG), as in Algorithm 2, the system generates vector embeddings for the entities in the refined graph. Let $E = \{e_1, e_2, \dots, e_{|E|}\}$ be the set of unique entities in the refined knowledge graph G . We define an embedding function $\phi: E \rightarrow \mathbb{R}^d$ that maps each entity textual representation to a dense vector. In our implementation, ϕ utilizes models such as llama3.3 or granite4-small-h, producing vectors of dimension d (e.g., $d = 4096$ or $d = 384$). The resulting set of vectors $\mathcal{V} = \{e_i \mid e_i = \phi(e_i), \forall e_i \in E\}$ is indexed using FAISS (IndexFlatIP) for efficient similarity search during retrieval.

H. Ontology Generation

Finally, the system analyzes the structure of the refined graph to infer classes and hierarchies, generating a formal OWL ontology. This ontology provides a structured schema for the extracted knowledge, facilitating interoperability and advanced reasoning capabilities.

I. GraphRAG Indexing and Querying

The system implements a Graph Retrieval-Augmented Generation (GraphRAG) engine to support complex queries over the constructed knowledge graph.

Given a natural language query Q , as in Algorithm 3, the system first computes its vector representation $\mathbf{q} = \phi(Q)$. To identify relevant entry points into the graph, we perform a

Algorithm 2: GraphRAG Indexing

Input : Merged Entities E , LLM Configuration C
Output : FAISS Index I , Entity Mapping M

- 1 Load entities from E ;
- 2 Initialize embedding model ϕ from C ;
- 3 $V \leftarrow \emptyset$;
- 4 $IDs \leftarrow \emptyset$;
- 5 **foreach** batch of entities $B \subseteq E$ **do**
- 6 $V_B \leftarrow \{\phi(e) | e \in B\}$;
- 7 Append valid embeddings from V_B to V ;
- 8 Append corresponding IDs to IDs ;
- 9 **if** checkpointing enabled **then**
- 10 Save intermediate state to disk;
- 11 Initialize FAISS index I (e.g., IndexFlatIP);
- 12 Add vectors V to I ;
- 13 Create mapping $M: IndexID \rightarrow EntityID$ using IDs ;
- 14 Save I and M to output directory;
- 15 **return** I, M

similarity search against the entity index. The set of entry nodes N_{entry} is defined as:

$$N_{entry} = \{e \in E \mid \text{sim}(\phi(e), \mathbf{q}) \geq \theta\} \cup \{e \in E \mid \text{keyword_match}(e, Q)\}$$

where $\text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a} \cdot \mathbf{b}}{\|\mathbf{a}\| \|\mathbf{b}\|}$ denotes cosine similarity. Starting from N_{entry} , we traverse the graph G to collect a subgraph of relevant triples $\mathcal{T}_{sub} \subset \mathcal{T}$ using a Breadth-First Search (BFS) limited to k hops.

To select the most relevant context for answer generation, we employ a graph-based scoring mechanism. Each triple $\tau \in \mathcal{T}_{sub}$ is linked to a source document d_τ . We define the relevance score of a document d as the frequency of its appearance in the retrieved subgraph:

$$\text{Score}(d) = |\{\tau \in \mathcal{T}_{sub} \mid d_\tau = d\}|$$

We identify the set of top M documents D_{top} with the highest scores. The final context triples $\mathcal{T}_{top}$ are selected from $\mathcal{T}_{sub}$ such that they belong to the documents in D_{top} . Crucially, we preserve the BFS traversal order in $\mathcal{T}_{sub}$, ensuring that triples closer to the query concepts are prioritized. The set is truncated to the top K triples:

$$\mathcal{T}_{top} = \{\tau \in \mathcal{T}_{sub} \mid d_\tau \in D_{top}\}_{1..K}$$

This ensures that the answer generation is grounded in the most structurally relevant publications while respecting the semantic proximity to the query.

V. EXPERIMENTS

We evaluate the system’s performance by processing a corpus of scientific publications. All experiments were conducted on a cluster with distributed inference servers.

A. Experimental Setup

The pipeline was configured to use multiple open-source LLMs hosted on vLLM and Ollama servers. The hardware infrastructure included distributed inference across four servers

Algorithm 3: GraphRAG Query Processing

Input : Query Q , Knowledge Graph G , FAISS Index I , Entity Mapping M , Configuration C
Output : Answer A , Sources S

// Step 1: Entity Extraction

- 1 $E_Q \leftarrow \emptyset$;
- 2 **if** Extractor LLM configured **then**
- 3 $E_Q \leftarrow LLM_{extractor}.extract(Q)$;
- 4 **else**
- 5 $E_Q \leftarrow KeywordExtract(Q)$;
- 6 // Step 2: Entry Point Discovery
- 6 $N_{entry} \leftarrow \emptyset$;
- 7 $\mathbf{q} \leftarrow \phi(Q)$;
- 8 **foreach** entity $e \in E_Q$ **do**
- 9 // Vector Search
- 9 $\mathbf{v}_e \leftarrow \phi(e)$;
- 10 $Indices \leftarrow I.search(\mathbf{v}_e, k=5)$;
- 11 $N_{entry} \leftarrow N_{entry} \cup \{M[idx] \mid idx \in Indices\}$;
- 11 // Keyword Search
- 12 $N_{entry} \leftarrow N_{entry} \cup \{n \in G.nodes \mid e \in n\}$;
- 12 // Direct Query Vector Search
- 13 $Indices_Q \leftarrow I.search(\mathbf{q}, k=5)$;
- 14 $N_{entry} \leftarrow N_{entry} \cup \{M[idx] \mid idx \in Indices_Q\}$;
- 14 // Step 3: Graph Traversal
- 15 $Triples \leftarrow \emptyset$;
- 16 $Docs \leftarrow \emptyset$;
- 17 **foreach** node $n \in N_{entry}$ **do**
- 18 Perform BFS traversal from n (depth
- 18 $k = C.k_hops$);
- 19 Collect visited edges as $Triples$;
- 20 Collect associated document IDs into $Docs$;
- 21 // Step 4: Hybrid Reranking
- 21 Calculate document scores $Score(d)$ based on $\mathcal{T}_{sub}$;
- 22 $D_{top} \leftarrow$ Select top M documents by $Score(d)$;
- 23 $\mathcal{T}_{top} \leftarrow \{\tau \in \mathcal{T}_{sub} \mid d_\tau \in D_{top}\}$ (retain BFS order);
- 24 $\mathcal{T}_{top} \leftarrow$ First K triples of $\mathcal{T}_{top}$;
- 24 // Step 5: Answer Generation
- 25 Construct context Ctx from $\mathcal{T}_{top}$ and D_{top} ;
- 26 $Prompt \leftarrow$ "Based on context Ctx , answer query $Q...$ ";
- 27 $A \leftarrow LLM_{reasoner}.generate(Prompt)$;
- 28 **return** A, D_{top}

with heterogeneous GPU capabilities. The server medz2 is equipped with an NVIDIA H100 GPU (80GB VRAM), representing the high-performance tier. The carz cluster nodes (carz1, carz2, carz3) utilize NVIDIA L40S GPUs (48GB VRAM). This distinction in hardware acceleration is a critical factor in the observed inference latencies. Wall-clock time and latency were measured to assess runtime performance.

B. Dataset

The corpus consists of 2,396 scientific documents stored in a MongoDB collection. These papers were taken from the

references of [31], and thus represent a broad sampling of the state of the neuromorphic field in 2017, permitting potential back-testing of this model against more recent trends in the community. This 2017 cutoff establishes a "temporal holdout" baseline to evaluate if GraphRAG can synthesize foundational concepts to answer queries about post-2017 developments. This suggests the system actively reasons over the structured KG rather than retrieving memorized recent literature from LLM pre-training. Currently, retrieval relies on graph topology and semantic similarity without explicit recency biasing. These documents are stored as plain text files. The corpus primarily consists of research papers in the domain of neuromorphic computing and neural network architectures. The text data serves as the input for the knowledge graph construction pipeline, where entities and relations are extracted to build the structured knowledge representation.

C. Runtime Performance

Table II summarizes the processing statistics. These timing details reflect the processing of a single prompt type (triple extraction) across the corpus.

TABLE II
CORPUS AND RUNTIME OVERVIEW. THESE ARE FOR ONLY ONE PROMPT
“TRIPLE_EXTRACTION” AND THE STAT FOR PROMPT
NEUROMORPHIC_EXTRACTION ARE NOT REPORTED HERE.

Metric	Value
Total Documents	2,396
Total Triples Extracted	515,615
Total Build Time	226h 47m 54s
Avg. Time per Document	340.8s
Overall Success Rate	100.0%

D. Model Specifications and Performance

We utilized four distinct models for extraction and reasoning, selected to evaluate performance across different parameter scales and quantization methods. Table III details the technical specifications of each model and its hosting environment.

Table III summarizes the runtime performance. The impact of hardware superiority is evident: despite the `gpt-oss-120b` model having the largest parameter count (120B), it achieved a significantly lower average latency (42.5s) on the H100-equipped `medz2` server compared to the smaller models running on L40S GPUs (e.g., `gpt-oss-20b` at 170.1s). This suggests that the H100’s advanced compute capabilities and memory bandwidth vastly outperform the L40S for these workloads, mitigating the cost of running larger models.

E. Extraction Yields and Reliability

The system achieved a 100% success rate across 10,572 queries, enabled by a robust regeneration mechanism for handling stochastic failures and malformed outputs. Regeneration is triggered only for empty responses, outputs under 50 characters, or JSON-like structures that fail validation, while cases where the model explicitly reports “no triples” are accepted without retry.

Because larger models incur substantial latency, the request timeout was set to 1200 secs. When regeneration occurs, the system attempts up to two retries, increasing the sampling temperature by 0.2 (capped at 0.8) and expanding the token limit by 10% per attempt. Network timeouts reuse the original parameters, and a fallback API protocol (e.g., OpenWebUI-style) is used as a last resort.

As shown in Table III, this strategy was essential for maintaining throughput. The `gpt-oss-20b` model was the least stable, with a regeneration rate of 111.5% and 12 timeouts despite the 1200secs limit. Nevertheless, 89.4% of failed attempts were successfully recovered, allowing the model to produce the largest number of triples. This highlights a key trade-off: smaller, less stable models can yield high extraction volume when supported by resilient, fault-tolerant error handling.

F. Graph Consolidation

After extraction, the merging and deduplication stages consolidated the raw triples into a unified graph. Out of 921,545 total triples, 515,615 were generated using the baseline triple-extraction prompt, while the remainder came from the neuromorphic-specific prompt designed to capture domain-level relations. Duplicate entities were identified and merged using fuzzy matching with a similarity threshold of 0.92. Subsequently, Louvain clustering with a minimum cluster size of 3 was applied to further refine the graph structure and reduce noise.

Figure 3 illustrates the final consolidated clean knowledge graph structure, showing a subgraph of the top 50 most frequent entities connected by 706 relationships.

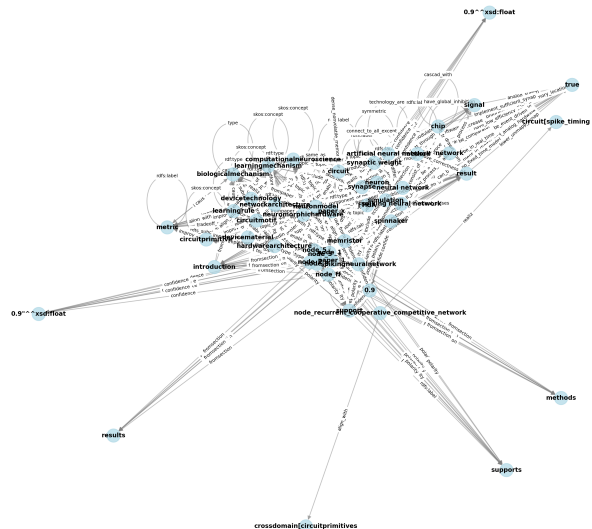


Fig. 3. Visualization of the final knowledge graph subgraph showing the top 50 most frequent entities and their relationships. Node labels represent entity names, and edges show the relations between them.

VI. GRAPHRAG USE CASE

The constructed KG serves as the foundation for a Graph Retrieval-Augmented Generation (GraphRAG) system.

TABLE III
MODEL SPECIFICATIONS, PERFORMANCE, AND RELIABILITY METRICS. REGENERATION RATE = (TOTAL REGENERATION ATTEMPTS / TOTAL REQUESTS) $\times 100\%$.

Host	GPU	Model	Params	Quant.	VRAM Usage	Requests	Average Latency	Triples	Regen. Rate	Regen. Success	Timeouts
medz2	H100	gpt-oss-120b	120B	MXFP4	~66GB / 81GB	2,651	42.5s	184,874	109.8%	90.2%	0
carz1	L40S	gpt-oss-20b	20B	MXFP4	~14GB / 46GB	2,646	170.1s	243,029	111.5%	89.4%	12
carz2	L40S	granite4-small-h	32B	Q4_K_M	~24GB / 46GB	2,639	7.5s	35,400	110.6%	90.0%	0
carz3	L40S	llama3.3-70b	70B	Q2_K	~42GB / 46GB	2,636	176.7s	52,312	112.7%	88.0%	0
–	–	Total	–	–	–	10,572	–	515,615	111.1%	89.4%	12

A. Query Processing Pipeline

The query mechanism follows a multi-stage process designed to retrieve contextually relevant information from the graph. First, the user’s natural language query is processed by an extractor LLM (e.g., gpt-oss-20b) to identify key entities. These entities are then mapped to graph nodes using a hybrid approach that combines vector similarity search on entity embeddings with exact keyword matching against node labels.

Starting from the identified entry nodes, the system performs a k-hop Breadth-First Search (BFS) (default $k=2$) to retrieve a relevant subgraph. This traversal captures the local neighborhood and structural context of the entities. The retrieved triples are then scored and ranked using a hybrid score that combines graph topology (node degree/centrality) and semantic relevance computed by a ranker LLM (e.g., granite4-small-h-ollama). Finally, the top-ranked triples and associated document context are fed into a reasoner LLM (e.g., gpt-oss-120b-ollama) to generate the final answer with citations.

B. Comparative Analysis

To evaluate the effectiveness of the GraphRAG system, we compared its performance against several “Naive” LLMs (i.e., zero-shot base models without RAG). The comparison utilized a specific query: “Which device technologies will dominate neuromorphic hardware beyond CMOS in the future?”.

We employed two primary metrics for this evaluation: **Perplexity** measures a model’s uncertainty or “surprise” when generating text; lower perplexity generally indicates higher fluency and confidence. **Semantic Similarity** computes the pairwise similarity between answers generated by different models to assess consensus and divergence.

Figure 5 illustrates the semantic agreement between the models. The RAG system shows distinct divergence from the smaller naive models, suggesting it retrieves unique, context-specific information not present in the models’ pre-trained weights. Figure 4 highlights the perplexity scores, where the

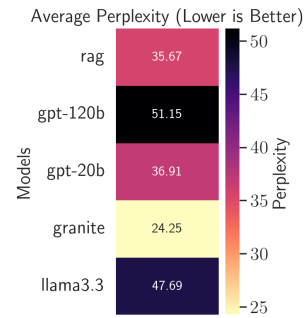


Fig. 4. Model perplexity comparison. Granite’s score appears lower due to short outputs. See Table V for comparison of responses from different LLMs.



Fig. 5. Semantic Agreement Matrix showing the similarity between answers from different models. Our proposed NEUKRAG shows distinct divergence.

RAG system achieves a competitive score (35.67), indicating that the augmented context does not degrade the fluency of the generation. Although the Granite model achieved the lowest perplexity in our evaluation, this result does not reflect higher answer quality. Its responses were very short and generic, often only a few sentences long. Perplexity is sensitive to output length and lexical diversity, so brief answers with simple phrasing naturally yield lower scores. In contrast, GraphRAG produced detailed, multi-paragraph explanations that incorporated retrieved KG facts, domain terminology, and multi-hop reasoning. This richer content increases token diversity and therefore raises perplexity, even while delivering more accurate and informative answers. Thus, the lower perplexity of Granite is primarily due to minimal output rather than stronger model performance.

While Perplexity and Semantic Similarity evaluate linguistic fluency, they do not measure engineering accuracy. GraphRAG’s “hardware-aware” capability is instead supported qualitatively (Table V), demonstrating retrieval of concrete physical constraints (e.g., energy per spike) from literature. This work focuses on foundational knowledge retrieval, not autonomous agentic execution (e.g., ReAct) for closed-loop simulations.

C. Qualitative Comparison

We evaluated seven queries spanning neuromorphic research—including learning rules beyond STDP, hybrid spiking–deep approaches, post-CMOS device technologies, hardware bottlenecks, memristor–CMOS comparisons, large-scale architectures, and biological trade-offs—to assess how different models reason about complex neuro-computational systems

(see Table IV). Across all queries, our GraphRAG system outperformed naive LLMs, producing responses that were longer (8,200 vs. 1,500 characters), more fluent (average perplexity 35.2 vs. 42.8), and technically deeper.

GraphRAG generated detailed, scientifically grounded explanations that integrated device physics, circuit behavior, architectural constraints, and learning theory, whereas naive LLMs offered shorter and more superficial answers. For example, on learning rules beyond STDP, GraphRAG produced a structured taxonomy of ten rules with hardware details, achieving $2.5\times$ lower perplexity and $6\times$ greater length than the best naive model. For device technologies beyond CMOS, it synthesized eight emerging technologies with clear neuromorphic relevance, far exceeding the naive models’ generic lists.

This advantage arises from GraphRAG’s ability to retrieve and integrate domain-specific knowledge from the curated KG, yielding context-rich, technically accurate responses resembling expert analysis. Naive LLMs, lacking structured neuromorphic grounding, frequently omitted key issues such as fabrication limits, scalability, and real-world implementations, resulting in incomplete or inaccurate reasoning.

Table V presents a snippet of the answers generated by the RAG system and the naive LLMs for the query: *Which device technologies will dominate neuromorphic hardware beyond CMOS in the future?* Table IV presents the performance metrics for these queries. The system provides not just an answer but also the supporting evidence (triples and source), enhancing trust and verification.

TABLE IV

GRAPHRAG QUERY PERFORMANCE FOR ALL 7 QUERIES (AFTER ENTITY EXTRACTION IMPROVEMENTS). THE TIME INCLUDES ENTITY EXTRACTION, SUBGRAPH RETRIEVAL, AND ANSWER GENERATION.

Query ID	Triples Retrieved	Query Time (in Secs)	Description
Q1	6796	40.1s	Learning rules beyond STDP
Q2	8344	40.0s	Hybrid learning paradigms
Q3	7249	40.2s	Device technologies beyond CMOS
Q4	8683	41.5s	Device technology competition
Q5	8630	42.2s	Hardware bottlenecks
Q6	6211	36.0s	Large-scale architectures
Q7	8256	36.7s	Biological vs engineering trade-offs

VII. CONCLUSION

This work presented an end-to-end framework for constructing neuromorphic Knowledge Graphs with lightweight, open-source LLMs. By distributing extraction across small models, the system performs high-fidelity concept mining without proprietary dependencies. The pipeline’s extraction, merging, and clustering stages yield a clean, deduplicated KG supporting richer reasoning than conventional dense retrieval. Through its GraphRAG layer, the system enables multi-hop explanations, explicit provenance, and domain-aware inference aligned with neuromorphic co-design needs.

The resulting KG unifies devices, circuits, architectures, learning rules, and applications, capturing how algorithms, neural dynamics, circuit motifs, and technologies interact.

TABLE V

COMPARISON OF LLM RESPONSES FOR THE QUERY: “WHICH DEVICE TECHNOLOGIES WILL DOMINATE NEUROMORPHIC HARDWARE BEYOND CMOS IN THE FUTURE?” (EXCERPT: WORDS 100–250). NEUKRAG USED GPT-OSS-120B-OLLAMA AS THE REASONER. OTHER MODELS ARE NON-RAG, ZERO-SHOT BASELINES. NEUKRAG EXCELS BY RETRIEVING CONCRETE PHYSICAL CONSTRAINTS (E.G., $\geq 10^9$ SYNAPSES/CM²) OMITTED BY NAIVE MODELS.

NEUKRAG(Reasoner: GPT-OSS-120B)

...2-D cross-bar geometry gives $\geq 10^9$ synapses/cm² (orders beyond CMOS SRAM). • Sub-10 ns programming, ≤ 10 fJ updates. • BEOL compatibility. • 10 nm RRAM in pilot production (Crossbar, Panasonic, Intel). • In-situ learning in 64×64 and 128×128 arrays. • Commercial products expected in 2026–2028. *Challenges:* device variability, endurance (10^6 – 10^8 cycles), sneak-path currents, mitigated by selector devices. Ongoing work includes selector engineering and HfO₂/TaO_x/TiO₂ materials.

GPT-OSS-120B

...Spintronic devices (MTJ, SOT, STNO) with sub-fJ switching, useful stochasticity, and synaptic/neuron roles. • MTJ and SOT devices demonstrated in Intel’s “Spin-Neuromorphic” chips (2023–24). • FeFET/FTJ devices with < 10 fJ updates and CMOS compatibility (Hf-based ferroelectrics). • FeFET arrays (IBM, GF) reaching > 1 M synapses at 7 nm. • Optical-neuromorphic systems offering light-speed interconnects.

Llama 3.3

(Full response, < 100 words) Beyond CMOS: memristors, PCM, STT-MRAM, graphene-based devices, and quantum-dot devices—enabling dense, low-power, high-speed neuromorphic hardware.

GPT-OSS-20B

...Photonic/optoelectronic neurons (THz bandwidth, negligible delay). • Quantum-dot/nanowire devices with intrinsic stochasticity. • Hybrid CMOS–MEMS actuators for low-power weight tuning. Trend toward hybrid stacks: CMOS control plus non-volatile analog synaptic devices. Most mature: memristors, PCM, spin-torque devices.

Granite 4

(Full response, < 100 words) Emerging devices include RRAM, PCM, MTJs, FeFETs, and optoelectronic components—offering dense analog computation. Remaining challenges: variability, reliability, and integration with CMOS at scale.

Grounding retrieval in this structure allows the RAG system to produce explanations that are consistent across scales and more faithful to neuromorphic dependencies than standalone LLM outputs.

Future work will expand and refine the KG via improved canonicalization, entity linking, and incremental updates, and strengthen KG and RAG evaluation using automated benchmarks. Future integration with autonomous agentic frameworks (e.g., ReAct) will enable LLMs to interface with simulators (SuperNeuro, NeuroCoreX) for closed-loop codesign. We also plan to expand the corpus with recent literature and implement recency biasing to predict future hardware trends. Ultimately, open LLMs, structured knowledge, and explainable retrieval provide a foundation for next-generation assistants supporting complex neuromorphic co-design.

REFERENCES

- [1] J. B. Aimone, "A roadmap for reaching the potential of brain-derived computing," *Advanced Intelligent Systems*, vol. 3, no. 1, p. 2000191, 2021.
- [2] A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, "Memory devices and applications for in-memory computing," *Nature nanotechnology*, vol. 15, no. 7, pp. 529–544, 2020.
- [3] A. R. Young, M. E. Dean, J. S. Plank, and G. S. Rose, "A review of spiking neuromorphic hardware communication systems," *IEEE Access*, vol. 7, pp. 135 606–135 620, 2019.
- [4] D. Kudithipudi, A. Daram, A. M. Zyarah, F. T. Zohora, J. B. Aimone, A. Yanguas-Gil, N. Soures, E. Neftci, M. Mattina, V. Lomonaco *et al.*, "Design principles for lifelong learning ai accelerators," *Nature electronics*, vol. 6, no. 11, pp. 807–822, 2023.
- [5] D. Kudithipudi, C. Schuman, C. M. Vineyard, T. Pandit, C. Merkel, R. Kubendran, J. B. Aimone, G. Orchard, C. Mayr, R. Benosman *et al.*, "Neuromorphic computing at scale," *Nature*, vol. 637, no. 8047, pp. 801–812, 2025.
- [6] J. B. Aimone, P. Date, G. A. Fonseca-Guerra, K. E. Hamilton, K. Henke, B. Kay, G. T. Kenyon, S. R. Kulkarni, S. M. Mniszewski, M. Parsa *et al.*, "A review of non-cognitive applications for neuromorphic computing," *Neuromorphic Computing and Engineering*, vol. 2, no. 3, p. 032003, 2022.
- [7] S. Ornes, "Can neuromorphic computing help reduce ai's high energy cost?" *Proceedings of the National Academy of Sciences*, vol. 122, no. 44, p. e2528654122, 2025.
- [8] C. D. Schuman, S. R. Kulkarni, M. Parsa, J. P. Mitchell, P. Date, and B. Kay, "Opportunities for neuromorphic computing algorithms and applications," *Nature Computational Science*, vol. 2, no. 1, pp. 10–19, 2022.
- [9] M. Davies, A. Wild, G. Orchard, Y. Sandamirskaya, G. A. F. Guerra, P. Joshi, P. Plank, and S. R. Risbud, "Advancing neuromorphic computing with loihi: A survey of results and outlook," *Proceedings of the IEEE*, vol. 109, no. 5, pp. 911–934, 2021.
- [10] H. A. Gonzalez, J. Huang, F. Kelber, K. K. Nazeer, T. Langer, C. Liu, M. Lohrmann, A. Rostami, M. Schöne, B. Vogginger *et al.*, "Spinnaker2: A large-scale neuromorphic system for event-based and asynchronous machine learning," *arXiv preprint arXiv:2401.04491*, 2024.
- [11] T. Dalgaty, F. Moro, Y. Demirağ, A. De Pra, G. Indiveri, E. Vianello, and M. Payvand, "Mosaic: in-memory computing and routing for small-world spike-based neuromorphic systems," *Nature Communications*, vol. 15, no. 1, p. 142, 2024.
- [12] S. Moradi, N. Qiao, F. Stefanini, and G. Indiveri, "A scalable multi-core architecture with heterogeneous memory structures for dynamic neuromorphic asynchronous processors (dynaps)," *IEEE transactions on biomedical circuits and systems*, vol. 12, no. 1, pp. 106–122, 2017.
- [13] C. Li, N. Imam, and R. Manohar, "A deterministic neuromorphic architecture with scalable time synchronization," *Nature Communications*, vol. 16, no. 1, p. 10329, 2025.
- [14] S. G. Cardwell, J. D. Smith, K. Patel, A. Maicke, J. Arzate, S. Liu, J. Kwon, C. R. Allemang, D. C. Crowder, S. Misra *et al.*, "Ai-guided codesign for novel computing paradigms," in *Proceedings of the 30th Asia and South Pacific Design Automation Conference*, 2025, pp. 849–856.
- [15] X. Wei, X. Cui, N. Cheng, X. Wang, X. Zhang, S. Huang, P. Xie, J. Xu, Y. Chen, M. Zhang *et al.*, "Chatie: Zero-shot information extraction via chatting with chatgpt," *arXiv preprint arXiv:2302.10205*, 2023.
- [16] A. Yates, M. Banko, M. Broadhead, M. J. Cafarella, O. Etzioni, and S. Soderland, "Textrunner: open information extraction on the web," in *Proceedings of Human Language Technologies: The Annual Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, 2007, pp. 25–26.
- [17] M. Agrawal, S. Heggelmann, H. Lang, Y. Kim, and D. Sontag, "Large language models are few-shot clinical information extractors," *arXiv preprint arXiv:2205.12689*, 2022.
- [18] M. Y. Jaradeh, A. Oelen, K. E. Farfar, M. Prinz, J. D'Souza, G. Kismihók, M. Stocker, and S. Auer, "Open research knowledge graph: Next generation infrastructure for semantic scholarly knowledge," in *Proceedings of the 10th international conference on knowledge capture*, 2019, pp. 243–246.
- [19] I. Beltagy, K. Lo, and A. Cohan, "Scibert: A pretrained language model for scientific text," *arXiv preprint arXiv:1903.10676*, 2019.
- [20] X. He, Y. Tian, Y. Sun, N. Chawla, T. Laurent, Y. LeCun, X. Bresson, and B. Hooi, "G-retriever: Retrieval-augmented generation for textual graph understanding and question answering," *Advances in Neural Information Processing Systems*, vol. 37, pp. 132 876–132 907, 2024.
- [21] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph rag approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.
- [22] S. Mudgal, H. Li, T. Rekatsinas, A. Doan, Y. Park, G. Krishnan, R. Deep, E. Arcaute, and V. Raghavendra, "Deep learning for entity matching: A design space exploration," in *Proceedings of the 2018 international conference on management of data*, 2018, pp. 19–34.
- [23] P. Date, C. Gunaratne, S. R. Kulkarni, R. Patton, M. Coletti, and T. Potok, "Superneuro: A fast and scalable simulator for neuromorphic computing," in *Proceedings of the 2023 International Conference on Neuromorphic Systems*, 2023, pp. 1–4.
- [24] J. B. Aimone, W. Severa, and C. M. Vineyard, "Composing neural algorithms with fugu," in *Proceedings of the International Conference on Neuromorphic Systems*, 2019, pp. 1–8.
- [25] J. E. Pedersen, S. Abreu, M. Jobst, G. Lenz, V. Fra, F. C. Bauer, D. R. Muir, P. Zhou, B. Vogginger, K. Heckel *et al.*, "Neuromorphic intermediate representation: A unified instruction set for interoperable brain-inspired computing," *Nature Communications*, vol. 15, no. 1, p. 8122, 2024.
- [26] A. Gautam, T. Kohno, P. Date, R. Patton, and T. Potok, "A suppression-based stdp rule resilient to jitter noise in spike patterns for neuromorphic computing," in *2024 International Conference on Neuromorphic Systems (ICONS)*. IEEE, 2024, pp. 209–216.
- [27] A. Gautam and T. Kohno, "Adaptive stdp-based on-chip spike pattern detection," *Frontiers in Neuroscience*, vol. 17, 2023.
- [28] A. Gautam, S. Kulkarni, R. Patton, T. Potok *et al.*, "Neurocorex: An open-source fpga-based spiking neural network emulator with on-chip learning," *arXiv preprint arXiv:2506.14138*, 2025.
- [29] I. Hossen, W. A. Borders, A. Madhavan, S. Joshi, P. M. Braganca, J. J. McClelland, B. D. Hoskins, and G. C. Adam, "Reram/cmos array integration and characterization via design of experiments," *Advanced Electronic Materials*, p. 2500203, 2025.
- [30] B. V. Benjamin, N. A. Steinmetz, N. N. Oza, J. J. Aguayo, and K. Boahen, "Neurogrid simulates cortical cell-types, active dendrites, and top-down attention," *Neuromorphic Computing and Engineering*, vol. 1, no. 1, p. 013001, 2021.
- [31] C. D. Schuman, T. E. Potok, R. M. Patton, J. D. Birdwell, M. E. Dean, G. S. Rose, and J. S. Plank, "A survey of neuromorphic computing and neural networks in hardware," *arXiv preprint arXiv:1705.06963*, 2017.